Project's High-Level Design

on

Marketing Planning Assistant Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Names & Enrolment Numbers:

Sr No	Student Name	Enrolment
1	Aditi Gaikwad	EN22CS301040
2	Akhilesh Tiwari	EN22CS301085
3	Alfez Khan	EN22CS301102
4	Akshat Sakalye	EN22CS301093
5	Aanchal Chaturvedi	EN22CS301014
6	Aditya Patidar	EN22CS301060

Group Name: Group 06D1

Project Number: AAI-06

Industry Mentor Name: Aashruti Shah (Program Director, Datagami)

University Mentor Name: Prof. Nishant Shrivastava

Academic Year: 2025-26

Table of Contents

1. Introduction
    a. Scope of the document
    b. Intended Audience
    c. System overview
2. System Design
    a. Application Design
    b. Process Flow
    c. Information Flow
    d. Technology Stack & Engineering Details
    e. Key Design Considerations & Fallbacks
3. Data Design
    a. Data Model
    b. Data Access Mechanism
    c. Data Retention & Migration
4. Interfaces (API Catalogue)
5. State and Session Management
6. Caching
7. Non-functional Requirement
    a. Security Aspects
    b. Performance Aspects
8. HLD vs LLD Comparison (Architectural evolution)
9. References

1 Introduction

The Marketing Planning Assistant Agent is an enterprise-grade, multi-tenant AI platform designed to autonomously generate, format, and manage data-driven marketing execution schedules. Instead of relying on a single AI model to write a generic response, this system utilizes an Agentic Workflow orchestrator to decompose high-level marketing goals into actionable tasks. By integrating Retrieval-Augmented Generation (RAG) with local vector databases, the system grounds its marketing strategies in user-provided context (such as competitor data and strategy PDFs) rather than relying solely on base Large Language Model (LLM) knowledge.

1.1 Scope of the Document

This document provides the High-Level Design (HLD) for the Marketing Planning Assistant Agent. It outlines the macro-architecture, technology stack, multi-tenant data isolation strategy, and the orchestration flow of the multi-agent system. It also details the engineering methodologies, fallback mechanisms, and error-handling strategies necessary to maintain system resilience. The design encompasses the React/Vite frontend client, the FastAPI backend, the MySQL relational database, and the ChromaDB vector storage components.

1.2 Intended Audience

This document is intended for Software Engineers, AI/ML Developers, and System Architects responsible for implementing and evaluating the Agentic AI system. It also serves as a comprehensive architectural guide for project mentors and academic evaluators.

1.3 System Overview

The system is an autonomous backend orchestrator designed to decompose high-level marketing goals into actionable tasks. It operates on a strict multi-tenant architecture, ensuring that data, generated plans, and uploaded context are securely isolated by the organization.

The core orchestration relies on specialized AI personas:

- **The Strategist Agent:** Retrieves context from the ChromaDB vector store and formulates a strategic approach.
- **The Execution Planner Agent:** Translates the high-level strategy into a detailed, actionable execution schedule using standard Markdown.
- **The Content Publisher Agent:** Transforms the Markdown schedule into web-ready semantic HTML, utilizing custom tools to prevent LLM formatting hallucinations.

These agents are orchestrated using CrewAI and LiteLLM, exposed via secure, asynchronous FastAPI REST endpoints, and consumed by a modern Tailwind CSS React frontend.

2 System Design

The Marketing Planning Assistant Agent is built on a decoupled, API-driven client-server architecture. It emphasizes modularity, separation of concerns, and strict multi-tenant data isolation.

2.1 Application Design

For an Agentic AI system, a standard MVC (Model-View-Controller) pattern does not perfectly fit because the core "business logic" is non-deterministic and driven by Large Language Models (LLMs). Instead, the system utilizes a Modular Architecture inspired by Clean Architecture principles. This structure separates the API transport layer from the Agent orchestration and the external tool integrations.

The application is divided into five logical layers:

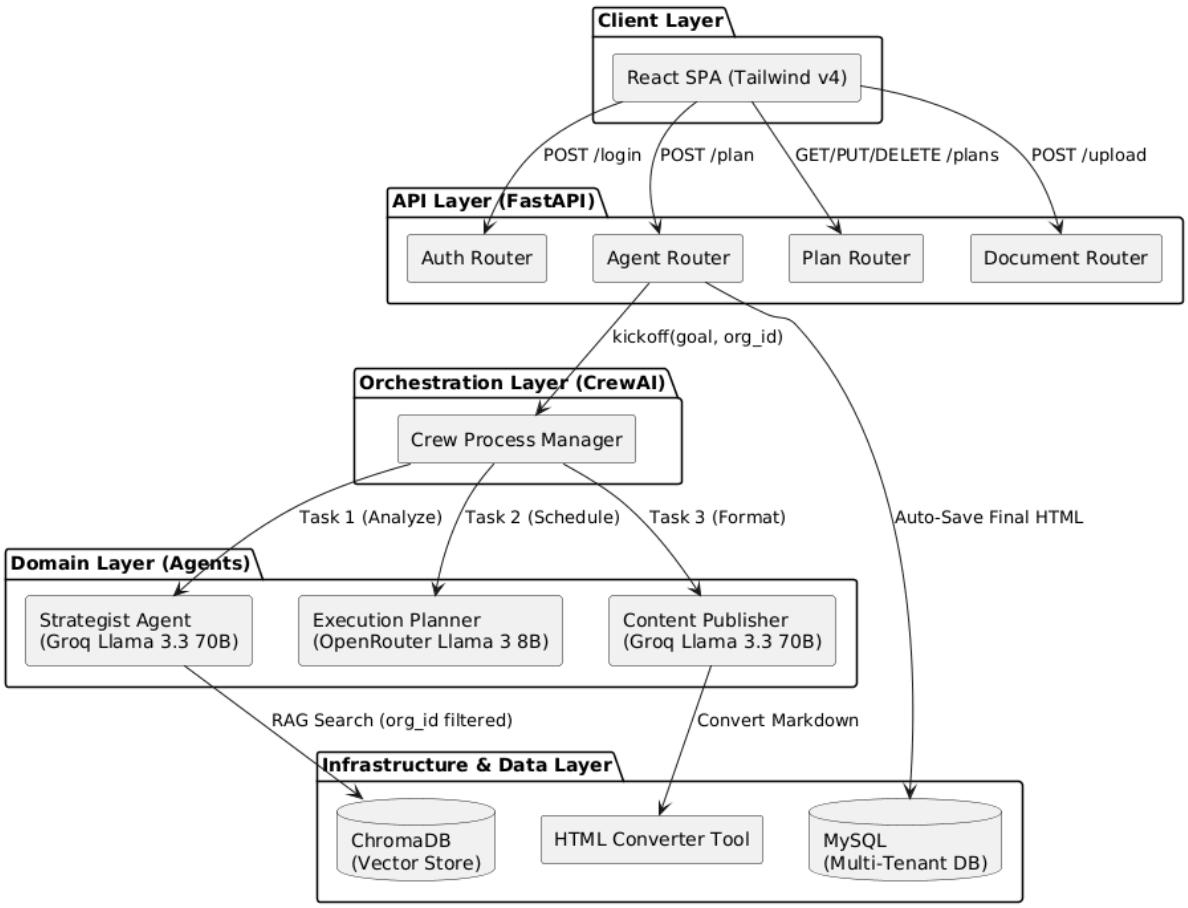
1. **API (Transport Layer):** Handles incoming HTTP requests from the React frontend, validates the JSON payload using Pydantic schemas, and routes objectives to the orchestrator.
2. **Orchestration (Service Layer):** Acts as the controller for the multi-agent CrewAI framework. It sequentially links the Agents to their respective Tasks.

3 **Agents (Domain Layer):** Contains the specific configurations, roles, goals, and backstories for the Strategist, Execution Planner, and Content Publisher.

4 **Tools (Infrastructure Layer):** Contains isolated, concrete implementations of custom tools (e.g., ChromaDB Vector Search, Markdown-to-HTML Converter) so developers can add new tools without touching core agent logic.

5 **Core (Configuration Layer):** Centralized utilities for database connection pooling, JWT security, and environment variable management.

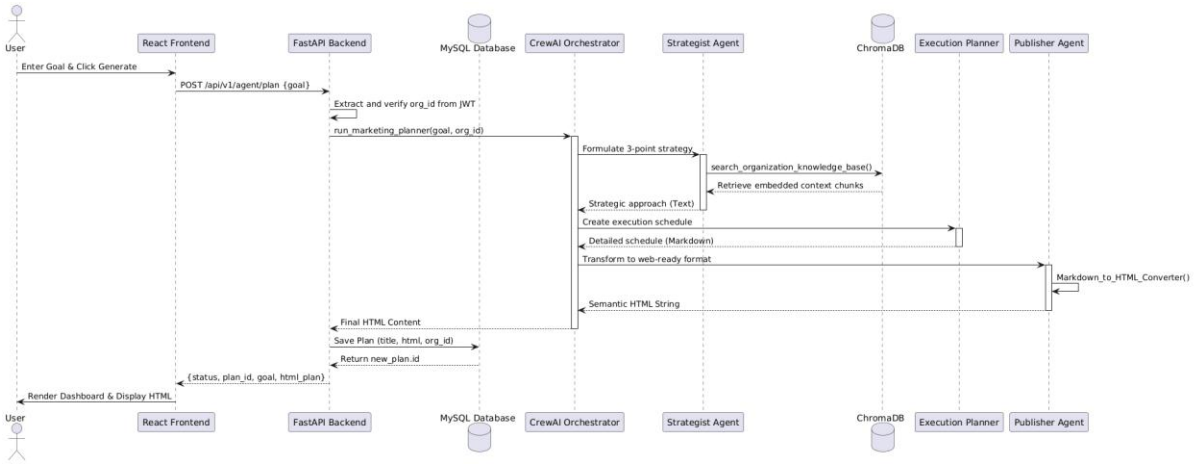
Component Architecture Diagram



2.2 Process Flow

The system employs a sequential Plan-and-Execute reasoning loop. The following sequence diagram maps out how a user's request travels through the API, interacts with the multi-agent LLM engine, invokes custom tools, auto-saves to the database, and returns formatted content to the UI.

Execution Sequence Diagram



2.3 Information Flow

Understanding how data transforms at each boundary is critical when bridging structured code and unstructured LLM outputs.

- **API Boundary:** All client-to-server and server-to-client communication strictly uses structured JSON validated by Pydantic and Axios.
- **Agent Boundaries:** Agents pass unstructured natural language and Markdown text between each other to maintain contextual reasoning.
- **Tool Boundaries:** Tools strictly accept required parameters and return strings or formatted JSON to prevent schema validation errors during LLM tool invocation.
- **Database Boundary:** The final output is transformed into structured relational rows in MySQL, explicitly tagged with an organization_id integer for tenant isolation.

2.4 Technology Stack & Engineering Details

To ensure rapid development, robust data validation, and seamless integration with Large Language Models, the following technology stack has been selected:

- **Engineering Type:** API-driven Micro-Monolith (Decoupled Client-Server architecture).
- **Development Model:** Agile, iterative prototyping with incremental feature rollouts.

- **Core Programming Language:** Python 3.10+ (Industry standard for AI orchestration).
- **Frontend:** React 18, Vite, Tailwind CSS v4, Lucide React, Axios.
- **Backend API Framework:** FastAPI & Uvicorn (Chosen for native asynchronous support, crucial for waiting on LLM network calls).
- **Agent Orchestration Framework:** CrewAI & LiteLLM (Provides high-level abstraction for managing multi-agent personas and routing requests to various LLM providers).
- **Database (Relational):** MySQL via SQLAlchemy ORM.
- **Database (Vector):** ChromaDB with HuggingFace sentence-transformers (all-MiniLM-L6-v2).
- **LLM Integration:** Groq (llama-3.3-70b-versatile) for heavy reasoning and strict formatting.
    - OpenRouter (meta-llama/llama-3-8b-instruct) for logistical scheduling.
- **Security & Environment:** PyJWT, bcrypt, and python-dotenv for secrets management.

2.5 Key Design Considerations & Fallbacks

Designing a non-deterministic AI system requires strict guardrails to ensure consistent output, handle provider outages, and maintain data integrity. The following design choices and fallback mechanisms have been implemented to guarantee system resilience.

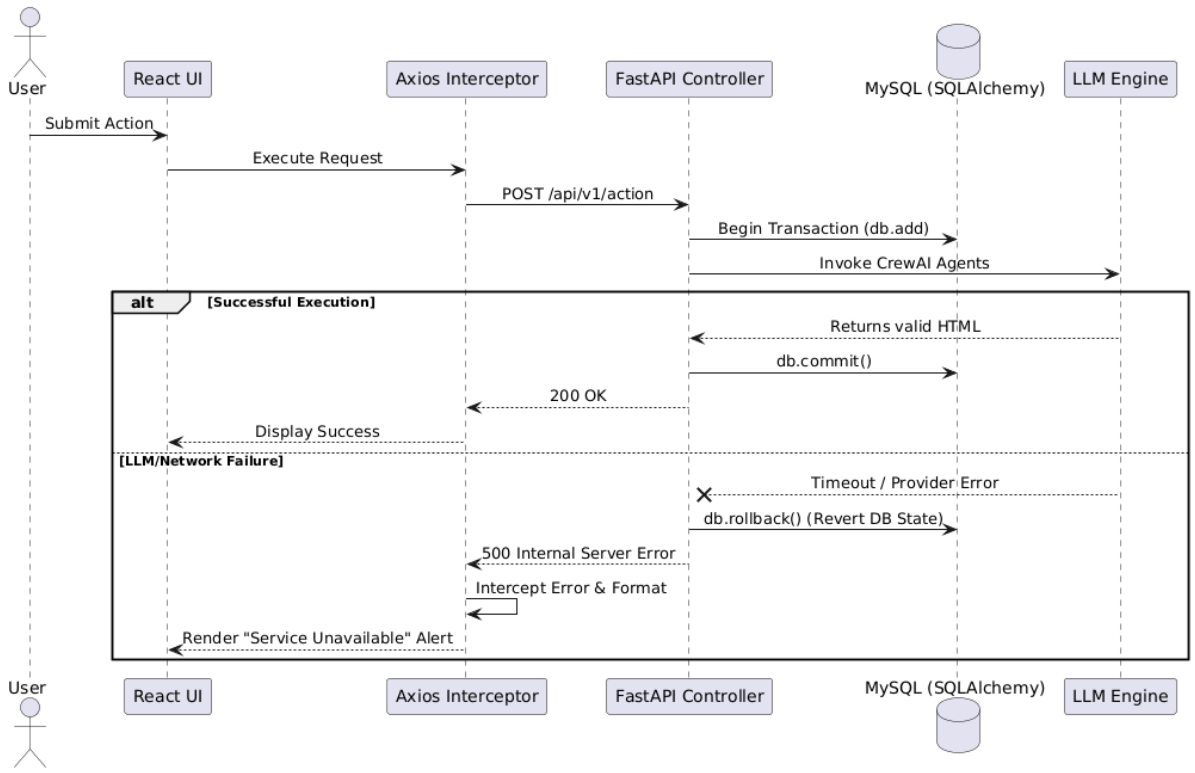
2.5.1 System Resilience & Error Handling

- **Atomic Database Transactions:** To prevent database corruption (e.g., creating a user without an organization, or saving a plan partially), the FastAPI backend utilizes strict SQLAlchemy transaction blocks. If an exception occurs during execution (e.g., the LLM generation fails midway), a db.rollback() is triggered, ensuring the database remains in a clean state.
- **Global Client-Side Interceptors:** The React frontend employs Axios interceptors to globally catch network failures (e.g., ERR_NETWORK when the FastAPI server

goes offline) and HTTP 500 errors. Instead of breaking the UI, these are mapped to user-friendly toast notifications, explicitly informing the user of the connection issue.

- **ORM Bidirectional Mapping:** Strict bidirectional back_populates definitions are enforced across SQLAlchemy models (e.g., Organization to Plan). This prevents ORM initialization KeyErrors and guarantees relational integrity when fetching tenant-specific history.

Error Handling & Rollback Flow Diagram



2.5.2 Fallbacks & Mitigation Strategies (Known Failure Scenarios)

Based on the development lifecycle, several critical failure scenarios were identified and mitigated:

1 **AI API Rate Limits & Decommissioned Models:**

- *Scenario:* Cloud AI providers frequently retire older models (e.g., mixtral-8x7b-32768) or impose strict rate limits, causing instant 400 Bad Request errors.

- *Mitigation:* The system utilizes a decoupled LLM Engine via LiteLLM. This allows the architecture to seamlessly swap decommissioned models for modern equivalents (e.g., llama-3.3-70b-versatile) by simply updating an environment variable or model string, requiring zero changes to the underlying agent orchestration logic.

2 **LLM Context Window Collapse (Formatting Failures):**

- *Scenario:* Smaller open-weight models (e.g., 8B parameters) can suffer from context collapse when fed massive walls of raw HTML, resulting in broken syntax (e.g., outputting just "} ") instead of the requested document.
- *Mitigation:* Task allocation relies on **Model Tiering**. Lightweight logistical tasks are routed to smaller models, while heavy string manipulation and structural translation (like the Content Publisher Agent) are strictly assigned to high-parameter models (70B+) with bulletproof, highly constrained expected_output system prompts.

3 **UI State Desynchronization (Dark Mode):**

- *Scenario:* Naive toggle functions in Single Page Applications (SPAs) can invert user preferences when navigating between virtual routes because the DOM does not refresh.
- *Mitigation:* The React useEffect hooks explicitly evaluate the browser's localStorage on component mount, forcefully applying or removing CSS classes rather than blindly toggling them. This guarantees persistent visual states across client-side navigation.

2.5.3 Prompt Engineering & Hallucination Mitigation

- **Separation of Concerns (Agents):** The system avoids relying on a single "God Agent". The Strategist handles pure logic, the Execution Planner handles scheduling, and the Content Publisher handles HTML formatting. This minimizes context window degradation and keeps LLMs focused on a single responsibility per turn.

- **Strict Tool Boundaries:** To prevent hallucinated formatting, agents are forbidden from writing raw HTML. They output standard Markdown, which is passed through a deterministic Python Markdown-to-HTML conversion tool. All tool interactions strictly exchange expected data types to prevent parsing errors.

2.5.4 Cost & Token Optimization

- **Multi-Provider Routing:** By orchestrating tasks through LiteLLM, the system dynamically routes generation requests to the most cost-effective provider. For example, utilizing OpenRouter for smaller formatting tasks while reserving Groq for high-speed, heavy-duty strategic reasoning optimizes both API expenditure and latency.

2.5.5 Observability & Logging

- **CrewAI Execution Trace:** The system maintains full visibility into the agentic loop. Backend logging captures the "scratchpad" reasoning of each agent, including tool invocations and intermediate Markdown generation, which is critical for debugging orchestration bottlenecks.

3 Data Design

The system employs a polyglot persistence strategy, utilizing both a relational database for structured application data and a vector database for unstructured contextual data. The paramount requirement of the data design is strict **Multi-Tenancy**—all records must be isolated by the user's Organization.

3.1 Data Model

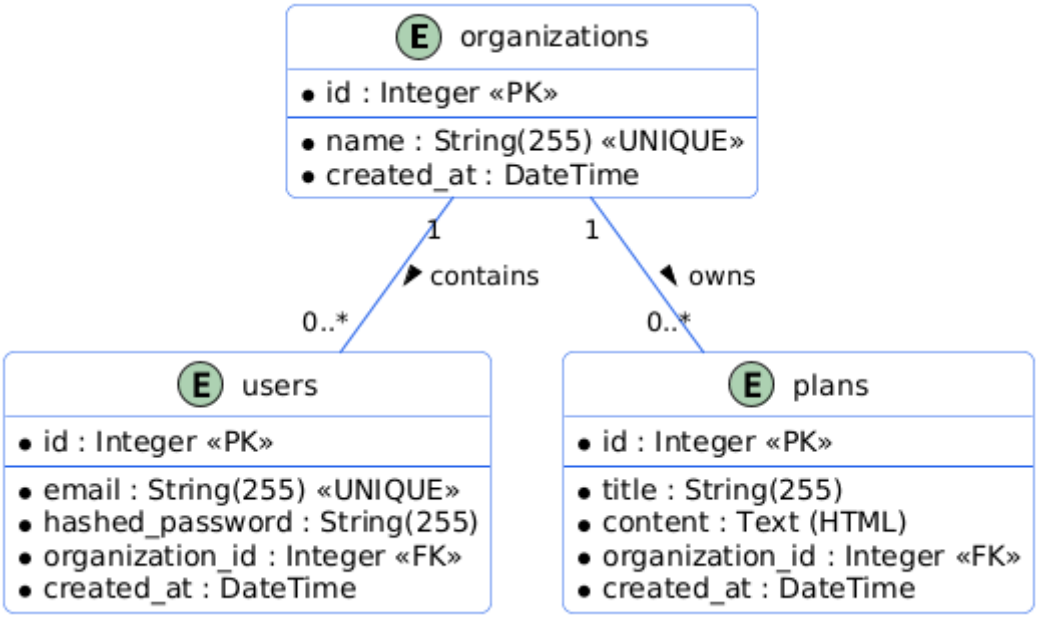
3.1.1 Relational Database (MySQL)

The relational schema is designed using a normalized structure to support tenant isolation and historical persistence.

- **Organization:** The root tenant entity.
- **User:** Represents individual accounts linked to an Organization.

- **Plan:** Stores the final Agentic outputs (HTML content, title, timestamp). It maintains a Many-to-One relationship with the Organization, ensuring users can only read, update, or delete plans belonging to their company.

Entity-Relationship Diagram (ERD)



3.1.2 Vector Database (ChromaDB)

Unstructured data (like uploaded competitor strategy PDFs) is parsed, chunked, and embedded using HuggingFace sentence-transformers (all-MiniLM-L6-v2).

- **Metadata Filtering:** Every embedded document chunk is stored in ChromaDB with metadata containing the org_id. During Retrieval-Augmented Generation (RAG), the Strategist Agent's search tool strictly filters by this org_id to prevent cross-tenant data leakage.

3.2 Data Access Mechanism

- **SQLAlchemy ORM:** The FastAPI backend interacts with MySQL exclusively through SQLAlchemy Object-Relational Mapping. This prevents SQL injection attacks and simplifies complex relational queries.

- **Pydantic Validations:** Data traversing the boundary between the API controllers and the ORM is strictly serialized and validated using Pydantic schemas (e.g., PlanUpdate, PlanResponse).

3.3 Data Retention & Migration

- **Auto-Save Protocol:** Plans are atomically saved to the database by the FastAPI controller *before* the API responds to the frontend. This guarantees zero data loss if the client disconnects immediately after generation.
- **Cascading Deletes:** Deleting an organization cascades to all child users and plans, ensuring GDPR compliance and preventing orphaned records.
- **Migrations:** Database schema migrations are handled via SQLAlchemy's Base.metadata.create_all() during initialization, with Alembic recommended for future production versioning.

4 Interfaces (API Catalogue)

The system is decoupled, communicating entirely over RESTful HTTP/JSON endpoints. All endpoints (except registration/login) require a valid Bearer JWT in the Authorization header.

Endpoint	Method	Payload / Request	Response / Purpose
/api/v1/auth/register	POST	{email, password, orgName}	Creates Organization & User. Returns JWT.
/api/v1/auth/login	POST	OAuth2PasswordRequestForm	Authenticates user. Returns JWT (access_token).

/api/v1/agent/plan	POST	{goal: "Analyze Q3 Ads"}	Triggers multi-agent Crew. Returns auto-saved Plan ID and HTML.
/api/v1/documents/upload	POST	multipart/form-data (PDF)	Vectorizes PDF and stores in ChromaDB with org_id metadata.
/api/v1/plans	GET	*None*	Returns array of PlanResponse objects for the user's tenant.
/api/v1/plans/{plan_id}	PUT	{title: "New Title"}	Renames a specific plan. Returns updated PlanResponse.
/api/v1/plans/{plan_id}	DELETE	*None*	Deletes a plan. Returns 204 No Content.

5 State and Session Management

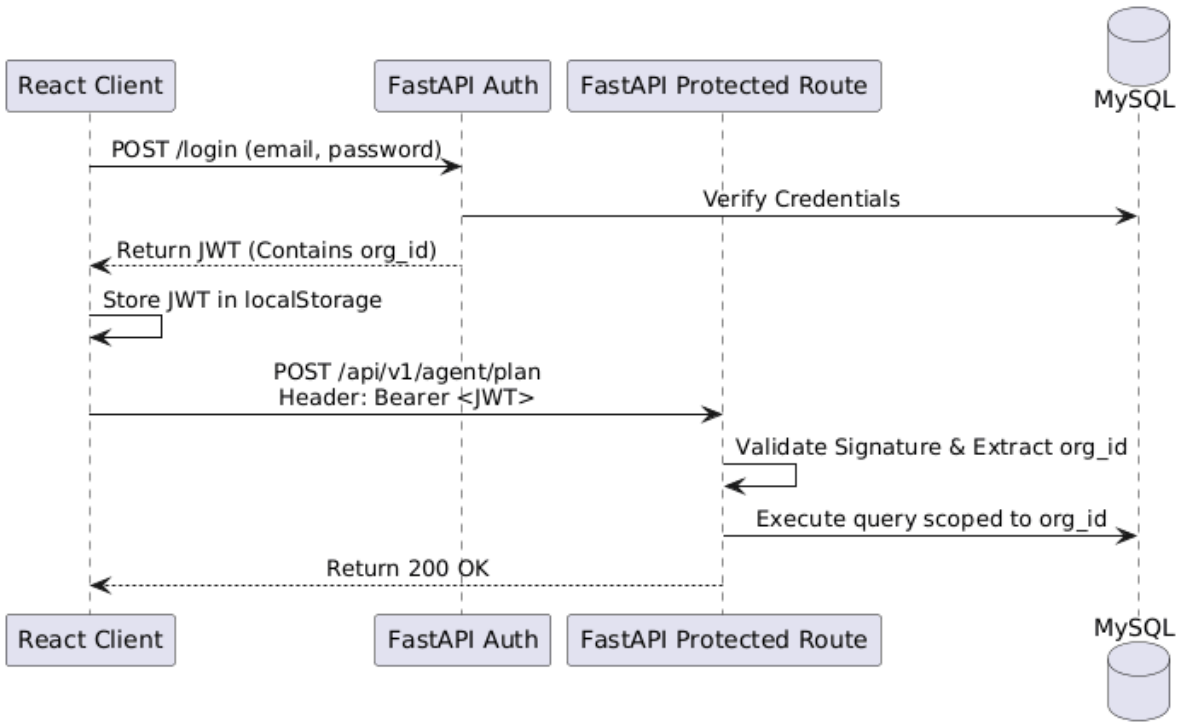
The architecture strictly adheres to stateless backend principles to ensure high horizontal scalability.

- **Client-Side State:** The React frontend manages local UI state (e.g., loading spinners, active modals) using React Hooks (useState, useEffect). Persistent user

preferences (like Dark/Light mode theme toggles) and the authentication token are stored securely in the browser's localStorage.

- **Stateless Backend (JWT):** The FastAPI backend does not store any session data in memory. Authentication is handled via JSON Web Tokens (JWT). Upon successful login, the server issues a JWT containing the user's org_id. The React client includes this token in the Authorization: Bearer <token> header of every subsequent request. The backend cryptographically verifies the token on the fly, extracting the tenant ID to authorize database and tool actions.

Stateless Authentication Flow



6 Caching

In this v1 architecture, aggressive backend caching (such as Redis) is not implemented. The primary latency bottleneck is the LLM inference generation time (typically 10-30 seconds), rather than database read/write speeds.

- **Future Enhancement:** As the user base grows, semantic caching (caching the LLM response for identical or highly similar marketing goals) can be introduced at the API gateway layer to reduce API costs and response times.

7 Non-functional Requirements

7.1 Security Aspects

- **Tenant Isolation:** Multi-tenancy is enforced at the database query level. No endpoint allows fetching records without explicitly filtering by the org_id derived from the validated JWT.
- **Secrets Management:** API keys (Groq, OpenRouter) and JWT secret keys are never hardcoded. They are managed via environment variables (.env) and injected at runtime.
- **Cryptographic Hashing:** User passwords are encrypted using bcrypt before being stored in MySQL.
- **Prompt Injection Defense:** User inputs (the marketing "goal") are sanitized and explicitly fenced within the CrewAI task definitions to prevent malicious instructions from hijacking the agent logic.

7.2 Performance Aspects

- **Asynchronous I/O:** FastAPI and Uvicorn utilize Python's asyncio to ensure that long-running LLM network calls do not block the main server thread, allowing the backend to handle concurrent users efficiently.
- **Model Tiering:** To optimize costs and token generation speed, lightweight formatting tasks are routed to smaller models (OpenRouter Llama 3 8B), reserving the heavier, more computationally expensive models (Groq Llama 3.3 70B) strictly for complex reasoning and strategic decomposition

8 HLD vs LLD Comparison (Architectural evolution)

The system design has significantly evolved from the initial Low-Level Design (LLD) specifications to the final production-ready High-Level Design (HLD). This evolution

reflects a shift from a theoretical mock-up to a fully functional, multi-tenant SaaS application.

Key Design Changes & Evolution:

1 **Frontend Paradigm Shift (Streamlit to React SPA):**
   a. *LLD Vision:* Proposed using Streamlit (or Gradio) for a rapid, Python-only UI.
   b. *HLD Evolution:* Upgraded to a **React 18 Single Page Application** powered by Vite and Tailwind CSS v4. This transition was necessary to support robust client-side routing, JWT session management, Dark/Light mode persistence, and a professional SaaS dashboard interface.

2 **Database & Multi-Tenancy Introduction:**
   a. *LLD Vision:* Did not specify persistent database storage or tenant isolation.
   b. *HLD Evolution:* Introduced a complete **Relational MySQL Database** managed via SQLAlchemy. This enabled strict multi-tenant data isolation, user authentication, and the Auto-Save capability for preserving historical marketing plans.

3 **Agent Roster Expansion:**
   a. *LLD Vision:* Defined a two-agent system: PlannerAgent and SchedulerAgent.
   b. *HLD Evolution:* Introduced a third agent, the **Content Publisher Agent**. This decoupled pure markdown generation from HTML structural formatting, preventing smaller LLMs from suffering context collapse when handling raw web code.

4 **Tooling Upgrade (Mocks to Real RAG):**
   a. *LLD Vision:* Relied on static "Mock" tools (e.g., BudgetValidatorMockTool, ResourceAvailabilityMockTool) returning hardcoded JSON responses.
   b. *HLD Evolution:* Replaced mock tools with a fully functional **Retrieval-Augmented Generation (RAG) pipeline**. Integrated ChromaDB and HuggingFace sentence-transformers, allowing the Strategist Agent to dynamically query real, user-uploaded PDF knowledge bases.

5 **LLM Provider Optimization:**

a. *LLD Vision:* Proposed using standard OpenAI or Gemini APIs.

b. *HLD Evolution:* Shifted to **Groq** and **OpenRouter** utilizing Meta's Llama 3 family of models. This strategic change leveraged Groq's LPU hardware for ultra-low latency inference, drastically reducing the 10-20 second execution time estimated in the LLD.

9 References

- **CrewAI Framework Documentation:** Used for multi-agent role-playing, task delegation, and execution orchestration.
- **FastAPI Documentation:** Used for building the asynchronous, high-performance RESTful API layer.
- **React & Tailwind CSS Documentation:** For utility-first frontend SPA development.
- **Project Low-Level Design (LLD) Report:** Group 06D1 , Academic Year 2025-26.